

GoComet Nova — Product Requirements Document (Part 1)

1. Understanding Nova

What is Nova? Nova is GoComet's shift from a supply-chain *dashboard* to a supply-chain *actor*. Traditional freight-forwarding SaaS shows a human that a Bill of Lading has a wrong HS code; Nova is meant to extract the field, cross-check it against the customer's rules, and hand a Cargo Group (CG) operator a ready decision and a ready reply — the human approves or edits, they don't originate the work. Traditional SaaS can't do this because the problem isn't storage or visibility, it's judgment applied to messy, unstructured documents at volume — that's an LLM-shaped problem, not a CRUD-shaped one.

What is the FDE model, and why here? Every enterprise customer's validation rules are different and mostly undocumented (a specific customer wants $\pm 0.5\%$ weight tolerance, another wants strict HS-code format, a third has a field that must appear identically across every document). A generic product can't ship "the right rules" out of the box — someone has to sit with the customer's actual process, encode it, and iterate when it's wrong. GoComet uses Forward Deployed Engineers to do that encoding directly (this project's `data/sample_docs/customer_rules/*.json` files are exactly that artifact: one file per customer, hand-authored from their requirements), rather than asking every customer to configure a generic rules UI themselves.

"System of Outcomes" vs. System of Record / Engagement. A System of Record answers "what happened" (a shipment log). A System of Engagement answers "how do people talk about it" (an inbox, a chat). A System of Outcomes answers "is the job actually done" — it takes the action, not just displays the data. Concretely in this build: the difference isn't cosmetic, it's a code-path one — the pipeline in this repo never just *renders* a validation result, it produces a `decision` and a `draft_email` as first-class stored outputs (`shipments.decision`, `shipments.draft_email`). The human still clicks send, but the *thinking and drafting* — the outcome — was Nova's job, not a report Nova generated for a human to redo.

2. Problem Statement

Where it breaks today. Trade documents (BOL, Commercial Invoice, Packing List, Certificate of Origin) are PDFs emailed around and checked by eye against rules that live in a CG operator's head or a spreadsheet.

1. **The omission** — a supplier forgets to attach the Packing List; a busy inbox means it isn't caught until customs rejects the shipment.
2. **The silent inconsistency** — Gross Weight reads `4,250 KG` on the BOL and `4.250 MT` on the Packing List. These are the same number in different units; a tired human either misses it or wastes time manually converting units to check.
3. **The undocumented rule violation** — a customer's SOP says Incoterms must be DDP; the supplier submits FOB; nobody catches it until the second amendment cycle, days later.

Success in the first 5 minutes. A CG operator opens the tool and sees a shipment already extracted, already cross-checked, with the one wrong field highlighted (found vs. expected, with the source snippet), and a drafted amendment email sitting below it, ready to edit and send. Zero PDFs opened by hand.

3. Users & Jobs-to-be-Done

Sarah — CG Operator. Validates every supplier submission against customer rules, all day. Amendment cycles (2–4 per shipment, 4–24 hours each) eat her bandwidth and her attention.

Michael — Shipping Unit (SU) coordinator. His job feels done the moment he hits send. Every extra amendment round is his company looking unreliable to the end customer.

1. **When** a shipment email/upload arrives, **I want** (Sarah) every attached document extracted and validated automatically, **so that** I never open a PDF just to find the one wrong field.
2. **When** a document has a discrepancy, **I want** (Sarah) to see exactly what was found vs. what was expected, with the confidence behind each, **so that** I trust the flag without re-checking the source myself.

3. **When** a field's extraction confidence is low or the scan is poor quality, **I want** (Sarah) the system to say so explicitly instead of guessing, **so that** I never approve something the system wasn't actually sure about.
4. **When** a shipment needs correction, **I want** (Sarah) a pre-drafted, specific amendment email, **so that** I edit and send instead of typing discrepancies out by hand.
5. **When** I submit documents, **I want** (Michael) fast, complete feedback on everything wrong at once, **so that** I fix it in one round instead of three back-and-forth emails.

4. Agent Architecture

Three LLM-backed agents, plus one deterministic (non-LLM) node, wired as a LangGraph state graph: `Extractor` → `Validator` → `Cross-Validate` (code, not a model) → `Router` → `Store`.

Why three agents, not one, not five

Option considered	Verdict
One giant prompt ("here's the PDF and the rules, decide and write the email")	Rejected. Conflates <i>reading</i> (probabilistic) with <i>policy enforcement</i> (must be deterministic and auditable). A single bad token breaks the whole outcome, and there's no seam to insert tool-calling for math a human could later audit line-by-line.
An agent per document type (separate BOL/Invoice/PackingList/CoO extractors)	Rejected. The difference between document types is <i>data</i> (which schema applies), not <i>reasoning</i> — handled inside one Extractor via <code>document_type</code> schema selection, not four prompts to maintain.
A fourth "Cross-Validator Agent" as its own LLM call	Rejected for now. Cross-document consistency (does the weight on the BOL match the Packing List, within tolerance?) is pure structural comparison — numeric-with-unit-conversion or fuzzy string ratio — not judgment. It's implemented as a plain Python graph node (<code>cross_validate_node</code> in <code>backend/pipeline/graph.py</code>), which is faster, free, and fully auditable (you can show a regulator the exact diff, not a model's opinion). Production note: this boundary should move to a real agent only if cross-doc rules become semantic (e.g. "does this Certificate of Origin logically support this Incoterm") rather than structural.
Three agents matching Extractor / Validator / Router (chosen)	Matches a planner/executor/verifier split: Extractor = executor (reads raw evidence), Validator = verifier (checks it against rules, with tool-calls for the parts that must be exact), Router = planner (decides the outcome and drafts the response).

Responsibility, input, output

- **Extractor** (`backend/agents/extractor.py`) — in: `PDF path + document_type` ; out: `ExtractionResult` — one `ExtractedField(value, confidence, source_snippet, page_number)` per required field (consignee, HS code, ports, Incoterms, description, weights, invoice number), never a bare value with no confidence.
- **Validator** (`backend/agents/validator.py`) — in: `ExtractionResult` + a customer's `CustomerRuleSet` JSON; out: `ValidationResult` with `match / mismatch / uncertain` per field. It's a **real tool-calling agent**: Claude decides *which* rule applies, but two deterministic Python functions — `evaluate_numeric_tolerance` (unit-aware: `KG↔MT↔LBS`) and `calculate_fuzzy_match` (`thefuzz` ratio) — do the actual arithmetic/string comparison as tool calls, so the model never does unit conversion "in its head."
- **Cross-Validate node** (deterministic, `graph.py`) — compares consignee, HS code, weights, Incoterms, ports, invoice number across every document in the shipment against a master value (BOL, or first doc), using the same numeric-tolerance/fuzzy logic as the Validator.
- **Router** (`backend/agents/router.py`) — in: `ValidationResult` + cross-doc result; out: `DecisionResult` (`approved / flagged / amendment_required`), a human-readable `reasoning` string, and a drafted email (LLM-written for amendments, templated for clean approvals — no LLM call needed when there's nothing to explain).

How agents talk to each other

Chosen: a single typed `PipelineState` (TypedDict) threaded through LangGraph nodes, serialized to plain dicts at each boundary so it's JSON-storable. **Considered and rejected:** a shared vector/blackboard memory (unnecessary non-determinism for a strictly sequential pipeline) and direct function calls passing raw Python objects between agents (loses the checkpointing and conditional-retry machinery LangGraph gives for free at each node boundary).

How state survives a crash — the honest version

What's real: every graph-node transition is persisted via a custom `SQLiteBackedMemorySaver` (`backend/pipeline/checkpoint.py`) that snapshots LangGraph's internal state into `nova.db`, plus an application-level `pipeline_checkpoints` row written on every stream tick. Verified by kill-9'ing the server mid-extraction and confirming the row survives.

What's not built yet, on purpose: there is no code path that re-enters `graph.stream()` from a saved checkpoint after a restart — a fresh run always starts clean. Given the time box, durable, provably-working checkpoint *writes* were the higher-value target than also building idempotent *resume* logic (which needs care: a resumed node must not silently re-bill an LLM call that actually succeeded before the crash).

Production plan: on startup, scan `pipeline_checkpoints` for shipments stuck in a non-terminal status, call `graph.get_state(config)` for that `thread_id`, and `graph.invoke(None, config)` to resume from the last completed node instead of re-extracting from scratch. Add a per-node idempotency marker so resuming never double-runs a node that already wrote its result.

5. LLM & Tooling Choices

Decision	Options considered	Chosen, and why
Which model per agent	Claude Sonnet everywhere (best reasoning, slower/pricier); Haiku everywhere (fast, cheap, iterate faster); a fixed mixed assignment	Model is a per-agent environment variable (<code>extraction_model</code> , <code>validation_model</code> , <code>routing_model</code> , <code>query_model</code> in <code>backend/config/settings.py</code>), not hardcoded — this project runs on Claude Haiku 4.5 across all four by default (<code>.env</code>), chosen to allow dozens of full-pipeline re-runs across 14 sample shipments and 3 inbox scenarios during development without burning budget or time on Sonnet's higher latency. Production plan: escalate per-document — Haiku by default, auto-retry the <i>specific</i> low-confidence document on Sonnet before falling back to human review, so the ~80% of clean scans stay cheap and the ~20% of hard cases get the stronger model. Not implemented yet — today every agent uses one fixed model, no dynamic escalation.
Vision input format	Rasterize every PDF to images always; only ever send native PDF	Native PDF (base64) for Anthropic models — preserves table/layout structure the model handles internally, skips a rasterization step entirely. Image fallback at 150 DPI (PyMuPDF) exists for two cases: non-Anthropic models, and automatic retry when the first extraction attempt fails to parse. 150 DPI was chosen empirically against the degraded scans (<code>shipment_4/9/12/13/14</code>): 72 DPI loses small print (HS codes, container numbers); 300 DPI roughly doubles token cost for no visible OCR gain on this sample set.
Orchestration framework	Hand-rolled sequential script; open-ended multi-agent frameworks (CrewAI/AutoGen-style); LangGraph	LangGraph. This pipeline is a strict DAG with one conditional retry edge, not an open-ended agent conversation — LangGraph's explicit node/edge graph is the right mental model, and its pluggable checkpointer interface let us add SQLite durability (see §4) without adopting a new datastore.
Structured output / tool use	Use it everywhere; avoid it everywhere	Used where correctness must be exact: Validator's numeric/fuzzy comparisons are real Anthropic tool calls into Python, not LLM arithmetic — a classic hallucination vector for compliance math is eliminated by construction. Router's decision is requested as <code>response_format=json_object</code> , but is backed by an unconditional deterministic fallback (<code>make_decision()</code>) that fires on any parse or API failure, so a single bad completion can never silently block the pipeline. Deliberately avoided in the Extractor: raw fields are extracted as free-form JSON

Decision	Options considered	Chosen, and why
		with a hand-written parser rather than a strict tool schema, because we need a nested union of 4 document schemas (BOL/Invoice/PackingList/CoO) and iterating on one flexible prompt was faster within the time box than maintaining four tool schemas. Production plan: once the four schemas stabilize, move extraction to Anthropic's schema-validated tool-use to eliminate JSON-parse failures entirely.
Query layer (text-to-SQL)	Parameterized query templates only (safe, but not real NL); LLM-generated SQL with no guardrail; LLM-generated SQL with a runtime guard	LLM-generated SQL, guarded server-side — <code>execute_query()</code> enforces a <code>SELECT</code> -only prefix and a regex block-list on <code>INSERT/UPDATE/DELETE/DROP/ALTER/CREATE/TRUNCATE</code> in code, not just as a prompt instruction, so even a confused model output can't mutate data. Production plan: replace the regex with a real SQL parser (e.g. <code>sqlglot</code>) to close edge cases like SQL comments or stacked statements, and execute against a database role that is read-only at the connection level as defense-in-depth.

6. Trust, Failure Handling & Evals

Stopping hallucination. The primary defense is not "ask the model if it's sure" — it's an independent check: `ocr_verifier.py` extracts raw text from the PDF via PyMuPDF (a completely separate code path from the vision extraction) and checks whether each LLM-claimed value actually appears in that text. Exact clean match → confidence unchanged; exact match but glued to adjacent text from a column-bleed → capped at 0.5; fuzzy partial match → capped at 0.75; not found at all → capped at 0.3 and flagged as a likely hallucination. Because this check doesn't ask the extractor to grade its own homework, it catches the failure mode where a confident-sounding model invents a plausible value.

No silent approval — the actual gate chain: (1) OCR-verifier confidence caps, (2) Validator marks any field below `confidence_threshold = 0.85` as `uncertain` without asking the LLM to judge its own low-confidence read, (3) a `document_quality_score < 0.6` forces the amendment path regardless of individual field results, (4) the Router's decision — whether from the LLM or its deterministic fallback — routes any `uncertain` field to at minimum `flagged`, never `approved`. Every layer is enforced in code, not prompt language alone.

Stopping runaway loops/cost. Extraction retries are capped at `max_retries = 2` (3 total attempts) inside the Extractor, plus one graph-level retry before routing to a terminal error state. Multi-document extraction runs via `asyncio.gather(..., return_exceptions=True)` so one bad document can't hang the rest of the shipment.

Honest gap: LangGraph's own `recursion_limit` isn't explicitly set today, and a `timeout_per_agent` setting exists in config but isn't wired into the actual LLM calls — dead configuration. **Production fix:** pass that timeout into every `litellm.completion(...)` call and set an explicit `recursion_limit` as a hard backstop independent of the app-level retry counters.

Evals — what exists vs. what we'd build next. Today: manual regression across 14 curated shipment scenarios (clean, deliberately mismatched, degraded scans) plus 3 inbox scenarios, re-run by hand after every prompt change — there is **no automated test suite** yet, which is the single biggest testing gap for taking this to production. **Offline eval to build first:** turn those 17 scenarios into a checked-in fixture set with an expected `decision` and expected flagged-field list per scenario, run as a CI regression job on every prompt/logic change. **Online metric to build next:** CG edit distance — the `PUT /api/shipments/{id}/send-amendment` endpoint already receives the operator's final edited text next to the AI's original `draft_email`, so a Levenshtein-distance metric between the two is a small addition to existing infrastructure, not new plumbing.

7. Metrics & Success Criteria

North-star: Straight-Through Processing (STP) rate — the share of shipments that reach `approved` with zero human edits to the draft and zero flagged/uncertain fields.

Supporting metrics (chosen because every one is already computable from the existing schema, not aspirational):
1. **Extraction confidence average** (`documents.extraction_confidence_avg`) — document/scan quality trend over time.
2. **Time-to-action** — `shipments.received_at` → `approve/send-amendment` timestamp (the same field powers the Part 2 north-star).
3. **Tokens & cost per document** (`documents.tokens_used`) — real per-run instrumentation, already stored, just not yet aggregated into a dashboard.
4. **False-negative rate** — auto-approved

shipments later found to have an error (target: 0%, checked against the golden-set fixtures above). 5. **Flagged/amendment rate by customer** — surfaces which customers' rules or suppliers need attention first. 6. **Cross-doc mismatch rate** — how often the deterministic cross-validate node actually fires; a proxy for how much value it's adding beyond per-doc validation.

Go/No-Go for a 2-week pilot: zero false negatives across the golden-set backtest; STP $\geq 20\%$ (1 in 5 shipments needs no human touch at all); CG-reported time-to-decision materially lower than their current baseline (self-reported, since pilot volume is too low for a clean statistical claim).

8. What's Next — Production Roadmap

If given the next two weeks, in priority order:

1. **Consume the checkpoints we already write** — implement the resume-from-checkpoint path described in §4, so a server crash mid-pipeline doesn't force a full re-run.
2. **Fix the cross-doc tolerance bug** — the cross-validate node currently hardcodes a 1% numeric tolerance instead of reading each customer's own `tolerance_pct` from their rules JSON (some customers, like HomeStyle Germany, define 0.5%). This is a real, found-in-testing gap, not hypothetical — cheap to fix, high trust impact.
3. **Enforce `present_on_all` rules** — the customer rule schema already supports this match type; the Validator currently parses but skips it. Closing this is what makes "the Packing List forgot the invoice number" actually get caught.
4. **Tiered model escalation** (Haiku default → Sonnet on low confidence) — see §5.
5. **Real inbound/outbound email plumbing** — today, even the "Send Amendment" button only updates a database status; there is no SMTP/mail-API integration at all, by design, to keep the human-in-the-loop boundary unambiguous during this build. Production needs a real inbound parser (e.g. AWS SES / Microsoft Graph inbound routing) and a real outbound send, still gated behind the same explicit CG click.
6. **Golden-set CI + edit-distance metric** — see §6.